

Clase 8: Aseguramiento de Calidad — Pruebas Web, API y Agentes con IA Generativa

Programa: Hardcore AI | 30X **Instructor:** [Por definir] **Duración total:** 2h (120 min) **Distribución:** 30% teoría (36 min) · 70% práctica (84 min) **Autor:** Andres Caicedo

Descripción

Estrategias y herramientas para el aseguramiento de calidad del producto, potenciadas por IA generativa. Esta sesión cubre los fundamentos del testing moderno — desde la pirámide de pruebas hasta BDD con Gherkin — y aborda tres dimensiones de pruebas: E2E web con Playwright, pruebas de API para servicios y abstracciones de agentes, y evaluación de agentes inteligentes usando el patrón Persona + Juez. El diferencial: cada paso se acelera con coding agents, MCPs de Playwright, y skills especializadas en IDEs agénticos.

Estructura del Módulo

BLOQUE TEÓRICO (36 min)

1. Apertura y Contexto (5 min)

- **2 min — Recapitulación** Repaso rápido: dónde estamos en el journey (producto implementado, pruebas unitarias/integración de sesiones anteriores). Hoy cerramos el ciclo de calidad.
- **3 min — El costo de no probar** Por qué las pruebas funcionales son el último filtro antes de producción. Casos reales de bugs que llegaron a usuarios por falta de pruebas en web, API y agentes.

2. La Pirámide de Pruebas (7 min)

- **4 min — El triángulo clásico**
 - Unitarias: rápidas, aisladas, validan lógica interna.
 - Integración: verifican que los componentes se comunican correctamente (API ↔ servicios, agente ↔ modelo).

- E2E: simulan el comportamiento real del usuario de punta a punta (web y API).
- Carga/Performance: validan comportamiento bajo estrés.
- **3 min — Dónde invertir el esfuerzo** Trade-offs entre velocidad, cobertura y costo de mantenimiento. El anti-patrón del "ice cream cone". Cómo distribuir esfuerzo entre pruebas web, API y agentes.

3. BDD, TDD y Gherkin (8 min)

- **2 min — TDD: Red-Green-Refactor** El ciclo que guía el diseño, no solo la verificación.
- **3 min — BDD: Comportamiento como especificación** Given/When/Then como lenguaje compartido entre negocio y desarrollo. BDD aplica tanto a flujos web como a contratos de API y conversaciones de agentes.
- **3 min — Gherkin: Features ejecutables** Anatomía de un `.feature` file. Cómo un escenario Gherkin se convierte en un test automatizado. Ejemplo concreto usando el proyecto ejemplo del curso.

4. Patrones de Diseño para Testing (4 min)

- **2 min — Page Object Model (POM)** Encapsular la estructura de cada página en una clase. Beneficios: mantenibilidad, reutilización, legibilidad.
- **1 min — Screenplay Pattern** Modelar tests como actores que realizan tareas. Ventajas sobre POM para flujos complejos y multi-actor.
- **1 min — Patrones para API testing** Request builders, response validators, y contract testing. Cómo estructurar tests de API para que sean mantenibles.

5. Pruebas de Agentes Inteligentes: Patrón Persona + Juez (6 min)

- **3 min — El problema de evaluar agentes** Los agentes producen respuestas no determinísticas. No basta con comparar strings. Se necesita evaluación semántica de calidad, relevancia y adherencia a instrucciones.
- **3 min — Arquitectura Persona + Juez**
- **Agente Persona:** Un LLM que simula un usuario real (user persona) e interactúa con el agente bajo prueba vía API, generando conversaciones realistas.
- **Agente Juez:** Un LLM evaluador que analiza la conversación completa y califica al agente en dimensiones como: precisión, tono, adherencia al contexto, manejo de edge cases, y alucinaciones.
- Flujo: Definir persona → Ejecutar conversación → Evaluar con juez → Generar scorecard.

6. El Stack de IA para Testing (6 min)

- **2 min — Coding agents como QA engineers** Cómo los agentes de código pueden generar, ejecutar y corregir tests de forma autónoma. El flujo: documentación BDD → Gherkin → Page Objects / API clients → Tests → Fix → Re-run.
 - **2 min — MCP de Playwright** Qué es el Playwright MCP server y cómo le da al agente la capacidad de interactuar con el browser en tiempo real: navegar, inspeccionar el DOM, tomar screenshots, y ejecutar acciones como un usuario.
 - **2 min — Skills y steering files para testing** Cómo usar skills y steering files en tu IDE agéntico (Kiro, Cursor, Claude Code) para que el agente siga automáticamente las convenciones de testing del proyecto: estructura de carpetas, naming, patrones POM, reglas de Gherkin, y convenciones de API testing.
-

BLOQUE PRÁCTICO (84 min)

7. Setup: Playwright + MCP + IDE Agéntico (10 min)

- **3 min — Instalación de Playwright** `npm init playwright@latest`, estructura de carpetas, `playwright.config.ts`. Configuración de browsers y viewports.
- **3 min — Configuración del Playwright MCP** Agregar el MCP server de Playwright al IDE agéntico. Verificar que el agente puede abrir el browser, navegar y tomar screenshots a través del MCP.
- **2 min — Skills de testing** Usar skills/steering files que definan las convenciones del proyecto: POM, estructura de carpetas `tests/e2e`, `tests/api`, `tests/agents`, naming conventions, y reglas de assertions.
- **2 min — Anatomía de un test Playwright** `test()`, `page`, `expect()`, locators, acciones básicas (`goto`, `click`, `fill`, `waitFor`).

8. Pruebas E2E Web con Asistencia de IA (22 min)

- **3 min — Demo: El agente escribe tu primer test web** Pedirle al coding agent que genere un test E2E para el flujo principal del proyecto ejemplo del curso. Observar cómo usa el MCP de Playwright para inspeccionar la app y generar locators precisos.
- **7 min — Hands-on: Primer test E2E web asistido** Cada participante usa su coding agent para generar un test que navegue al proyecto ejemplo, realice la acción principal y verifique el resultado. Ejecutar con `npx playwright test`, revisar el reporte HTML, y iterar con el agente para corregir fallos. Explorar también el modo interactivo con `npx playwright test --ui` para visualizar, filtrar y re-ejecutar tests desde el navegador.

- **3 min — Demo: Generación de Page Objects desde el DOM** Mostrar cómo el agente usa el Playwright MCP para inspeccionar el DOM del proyecto ejemplo y generar Page Objects automáticamente con locators correctos.
- **9 min — Hands-on: Refactor a POM con el agente** Pedirle al coding agent que refactorice los tests a Page Object Model. Crear al menos 2 page objects. Verificar que los tests siguen pasando.

9. Pruebas de API: Servicios y Abstracciones de Agentes (18 min)

- **3 min — Demo: Tests de API con Playwright** Usar `request` context de Playwright para probar endpoints REST del proyecto ejemplo. Validar status codes, response bodies, headers y contratos. Mostrar cómo el agente genera tests de API a partir de la documentación de endpoints.
- **8 min — Hands-on: Suite de pruebas de API** Cada participante usa su coding agent para generar tests de API para al menos 3 endpoints del proyecto ejemplo: happy path, error handling, y validación de contratos. Ejecutar y verificar resultados.
- **3 min — Demo: Probando abstracciones de agentes vía API** Cómo testear los endpoints que exponen agentes inteligentes: enviar prompts, validar estructura de respuesta, verificar que el agente respeta su system prompt y contexto.
- **4 min — Hands-on: Test de agente vía API** Escribir un test que envíe una consulta al endpoint del agente del proyecto ejemplo y valide que la respuesta cumple con el contrato esperado (formato, campos, coherencia básica).

10. Evaluación de Agentes: Patrón Persona + Juez (16 min)

- **4 min — Demo: Implementando el patrón** Mostrar la implementación del flujo completo sobre el proyecto ejemplo: 1. Tomar los user personas ya definidos en la documentación del proyecto (perfil, intención, estilo de comunicación). 2. Script que usa un LLM como agente persona para generar una conversación multi-turno con el agente bajo prueba vía API. 3. Script que usa un LLM como juez para evaluar la conversación con un rubric predefinido (precisión, tono, alucinaciones, adherencia al contexto). 4. Generar un scorecard con resultados.
- **12 min — Hands-on: Evaluar el agente del proyecto ejemplo** Cada participante ejecuta el flujo Persona + Juez contra el agente del proyecto ejemplo, usando al menos 2 de los user personas existentes en la documentación. Ejecutar conversaciones, evaluar con el juez, y comparar scorecards. Iterar con el coding agent para ajustar el rubric de evaluación.

11. BDD: De Gherkin a Tests Automatizados (10 min)

- **2 min — Setup de playwright-bdd** Configuración rápida de `playwright-bdd` para conectar archivos `.feature` con Playwright.

- **3 min — Demo: De las historias de usuario a archivos .feature** El proyecto ejemplo ya cuenta con documentación en markdown que incluye historias de usuario y escenarios BDD. Usar el coding agent para transformar esos escenarios en archivos `.feature` ejecutables con sintaxis Gherkin, manteniendo la trazabilidad con la documentación original.
- **5 min — Hands-on: Feature file → Step definitions → Ejecución** Escribir un `.feature` con 2-3 escenarios del proyecto ejemplo. Pedirle al agente que genere los step definitions usando los Page Objects ya creados. Ejecutar y ver los escenarios pasar.

12. Modos de Ejecución y Paralelismo (4 min)

- **1 min — Multi-browser y versiones** Configurar `projects` en `playwright.config.ts` para ejecutar tests en Chromium, Firefox y WebKit.
- **1 min — Headed vs Headless** Diferencias entre modo headed (visual, debugging) y headless (rápido, CI). `--headed` para desarrollo, headless por defecto en pipelines.
- **2 min — Paralelismo con workers** Cómo Playwright ejecuta tests en paralelo con workers. Configuración de `workers` en config y CLI (`--workers=4`). Trade-offs: velocidad vs recursos vs aislamiento de estado.

13. Debugging y Evidencia Automatizada (4 min)

- **2 min — Herramientas de debugging** Playwright Inspector, trace viewer, screenshots on failure, video recording. Cómo el agente puede interpretar trases y sugerir fixes.
- **2 min — Hands-on: Configurar trace y screenshot on failure** Configurar el proyecto para capturar evidencia automática en cada fallo.

Cierre (8 min de los 84 prácticos)

- **3 min — Resumen:** Pirámide → BDD → Web E2E → API → Agentes (Persona + Juez) → Playwright → POM → MCP → CI. El flujo completo de calidad acelerado por IA.
 - **3 min — Reflexión:** ¿Qué cambió al usar el agente vs escribir tests manualmente? ¿Dónde el agente acertó y dónde necesitó corrección? ¿Cómo evaluar algo no determinístico?
 - **2 min — Q&A y troubleshooting** de los ejercicios.
-

Prerrequisitos

- Producto implementado (Clase 6).
- Node.js instalado.
- IDE agéntico configurado (Kiro, Cursor, o Claude Code con MCP habilitado).

- Familiaridad básica con terminal y TypeScript/JavaScript.

✓ Tarea

1. Aplicar lo aprendido en clase a tu propio producto: implementar al menos 3 tests E2E web y 3 tests de API con Playwright, usando Page Object Model, generados y refinados con tu coding agent.
2. Implementar el patrón Persona + Juez para evaluar el agente de tu producto, usando los user personas de tu documentación.
3. Escribir al menos 1 feature file en Gherkin con sus step definitions automatizados, partiendo de la documentación BDD existente.
4. Configurar Playwright para generar reportes HTML y traces on failure.

✂ Artefactos Esperados

Artefacto	Formato
Tests E2E web	tests/e2e/*.spec.ts
Page Objects	tests/pages/*.ts
Tests de API	tests/api/*.spec.ts
Tests de agentes (Persona + Juez)	tests/agents/*.spec.ts
Rubric de evaluación	tests/agents/rubrics/*.md
Feature files (Gherkin)	tests/features/*.feature
Step definitions	tests/steps/*.ts
Configuración Playwright	playwright.config.ts
Reporte HTML de ejecución	playwright-report/
Skill de testing	.kiro/skills/testing.md o equivalente
Configuración MCP Playwright	.kiro/settings/mcp.json o equivalente



Herramientas de IA Utilizadas

Herramienta	Uso en la sesión
Playwright MCP Server	El agente navega, inspecciona DOM, toma screenshots y ejecuta acciones en el browser
Playwright API Testing	<code>request</code> context para pruebas de endpoints REST y agentes vía API
Coding Agent (Claude Code / Kiro / Cursor)	Genera tests web, API, page objects, step definitions y corrige fallos
LLM como Agente Persona	Simula usuarios reales (desde documentación existente) interactuando con el agente bajo prueba
LLM como Agente Juez	Evalúa conversaciones con rubric predefinido y genera scorecards
Skills / Steering Files	Encapsulan convenciones de testing para que el agente las siga consistentemente